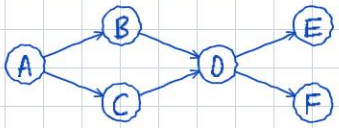
Breadth-First Search (BFS)

BFS is a graph traversal algorithm that explores vertices level by level - it radiates outward from a source node, visiting all nodes at distance 1 before distance 2, and so on.

Key Idea: Use a FIFO queue. When we visit a node, we enqueue all its unvisited neighbors. The queue guarantees we process nodes in order of increasing distance.

Example Graph (directed):



Algorithm Steps:

1. Initialize: mark source as visited → enqueue source
2. While queue is not empty:
   a. Dequeue node v (front of queue)
   b. For each neighbor u of v:
      → If u is unvisited: mark visited, enqueue u
3. Repeat until queue is empty

BFS Traversal from A - Queue States:

Step 1 - Enqueue source A

Step 2 - Dequeue A → - enqueue B, C

Step 3 - Dequeue B → enqueue D

Step 4 - Dequeue C → D already visited, skip

Step 5 - Dequeue D → . enqueue E, F

Step 6 - Dequeue E, then F → Queue empty, done!

Visit order: A → B → - C → D → E → F

BFS Tree (Levels):

Level 0: A
Level 1: B, C
Level 2: D
Level 3: E, F

{"type": "tree", "nodes": "A:B:C B:D:- C:-:- D:E:F E:-:- F:-:-"}

Applications:
- Shortest path in unweighted graphs (fewest edges between two nodes)
- Level-order traversal of trees
- Connected components + bipartiteness checking
- Web crawling - discover pages by increasing distance

Time Complexity: O(V + E)
- Each vertex is enqueued/dequeued at most once → · O(V)
- Each edge is examined once → - O(E)
- Space: O(V) for the queue + visited set

BFS explores level by level - use a queue